Experiment-10

Aim: Write a program to demonstrate continuous file allocation.

Description:

In the **contiguous file allocation technique**, each file occupies a single, unbroken sequence of adjacent blocks on a storage disk

Algorithm:

1. Start and initialize all 100 disk blocks to 0 (free).
2. Read the number of files and create an array of file records.
3. For each file, read its ID, name, starting block, and number of blocks.
4. Set the availability flag to 1 and examine consecutive blocks from start to start + length - 1.
5. If a block is occupied, set the flag to 0 and stop checking; otherwise, mark that block with the file ID.
6. If the flag is 0, display "blocks are not available" and terminate; otherwise, process the next file.
7. Display each file's ID, name, start, length, and consecutive allocated block numbers.
8. Stop.

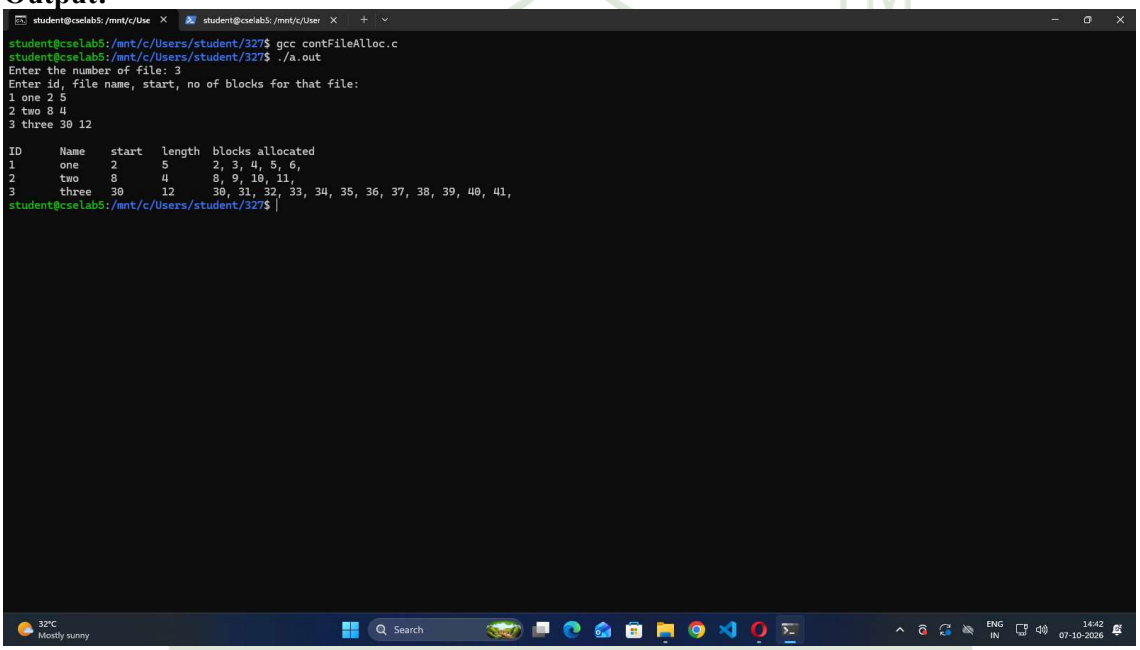
Code:

```
#include<stdio.h>

#define SIZE 100
int disk[SIZE] = {0};
struct file{
        int id;
        char name[20];
        int start;
        int length;
};
int main(){
        int n;
        printf("Enter the number of file: ");
        scanf("%d", &n);
        struct file files[n];
        printf("Enter id, file name, start, no of blocks for that file: \n");
        for(int i =0;i<n;i++){
                scanf("%d %s %d
%d",&files[i].id,files[i].name,&files[i].start,&files[i].length);
                int avail = 1;
                for(int j=files[i].start; j< files[i].start+files[i].length ;
j++){
                        if(disk[j] != 0){
                                avail = 0;
                                break;
                        }
                        disk[j] = files[i].id;
                }
                if(avail == 0){
                        printf("blocks are not available\n");
                        return 0;
                }
        }
        printf("\nID\tName\tstart\tlength\tblocks allocated\n");
        for(int i =0; i<n;i++){
                printf("%d\t%s\t%d\t%d\t",files[i].id,files[i].name,files[i].start
,files[i].length);
                for(int j= files[i].start; j< files[i].start+files[i].length;
```

```
j++){
                        printf("%d, ", j);
                }
                printf("\n");
        }
        return 0;
}
```

Output:



Aim: Write a program to demonstrate linked list file allocation technique.

Description:

In **linked list file allocation**, the operating system stores a file as a scattered linked list of disk blocks across the storage device, with each block containing a pointer to the next block in the chain

Algorithm:

1. Start, initialize all disk blocks to 0, and set each nextBlock entry to -1.
2. Read the number of files, then read each file's ID, name, starting block, and length.
3. If the start is outside 0 to 99, the length is nonpositive, or the ID is 0, display "Invalid file details" and terminate.
4. If the starting block is occupied, display an error and terminate; otherwise, allocate it and set the block count to 1.
5. While more blocks are needed, search from block 0 for the first free block. If none exists, display an error and terminate.
6. Allocate the free block, link it to the current block using nextBlock, move to it, and increment the count. Repeat for all files.
7. Display each file's details and follow nextBlock from its start to -1 to print the allocated chain.
8. Stop.

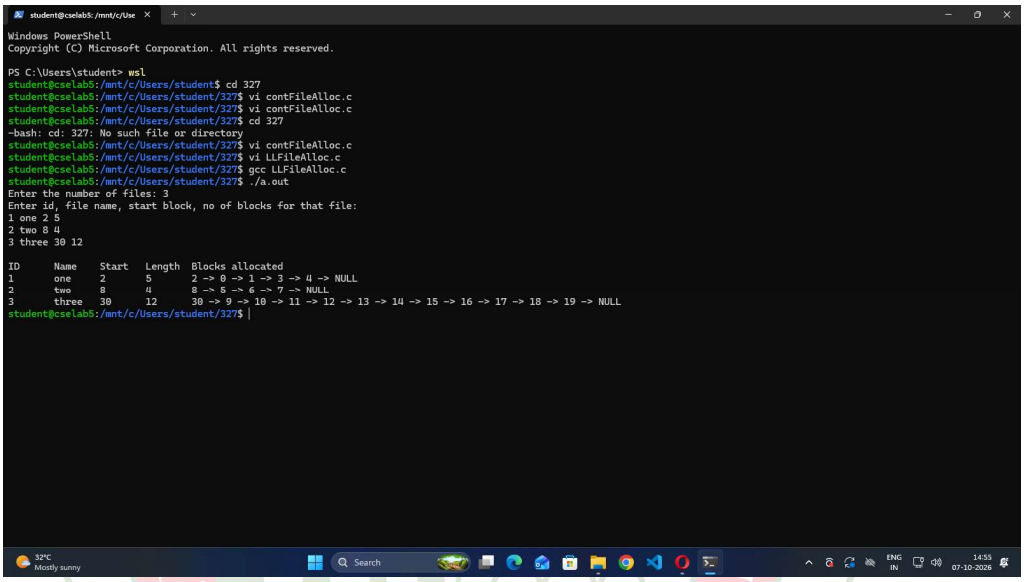
Code:

```
#include<stdio.h>
#define SIZE 100
int disk[SIZE] = {0};
int nextBlock[SIZE];
struct file{
    int id;
    char name[20];
    int start;
    int length;
};
int main(){
    int n;
    printf("Enter the number of files: ");
    scanf("%d", &n);
    struct file files[n];
    for(int i=0;i<SIZE;i++){
        nextBlock[i] = -1;
    }
    printf("Enter id, file name, start block, no of blocks for that file:\n");
    for(int i=0;i<n;i++){
        scanf("%d %19s %d
%d",&files[i].id,files[i].name,&files[i].start,&files[i].length);
        if(files[i].start < 0 || files[i].start >= SIZE || files[i].length <= 0 ||
files[i].id == 0){
            printf("Invalid file details\n");
            return 0;
        }
        int current = files[i].start;
        if(disk[current] != 0){
            printf("Starting block is not available\n");
            return 0;
        }
        disk[current] = files[i].id;
        int count = 1;
        while(count < files[i].length){
            int next = -1;
            for(int j=0;j<SIZE;j++){
                if(disk[j] == 0){
                    next = j;
                    break;
                }
            }
            if(next == -1){
                printf("Blocks are not available\n");
                return 0;
            }
            disk[next] = files[i].id;
            nextBlock[current] = next;
            current = next;
            count++;
        }
    }
    printf("\nID\tName\tStart\tLength\tBlocks Allocated\n");
    for(int i=0;i<n;i++){

printf("%d\t%s\t%d\t%d\t",files[i].id,files[i].name,files[i].start,files[i].length
);
        int current = files[i].start;
        while(current != -1){
```

```
            printf("%d",current);
            current = nextBlock[current];
            if(current != -1){
                printf(" -> ");
            }
        }
        printf("\n");
    }
    return 0;
}
```

Output:



Aim: Write a program to demonstrate indexed file allocation technique.

Description:

Indexed file allocation is an operating system storage management technique where each file is assigned a dedicated index block that stores the disk addresses of all the blocks containing the file's actual data.

Algorithm:

1. Start and initialize all 100 disk blocks to 0 (free).
2. Read the number of files, then read each file's ID, name, index block, and number of data blocks.
3. If the index is outside 0 to 99, the length is outside 1 to 99, or the ID is 0, display "Invalid file details" and terminate.
4. If the index block is occupied, display an error and terminate; otherwise, reserve it using the file ID.
5. For each required data block, search from block 0 for the first free block. If none exists, display an error and terminate.
6. Store the free block number in the file's blocks array and mark the disk block with the file ID. Repeat for all files.
7. Display each file's ID, name, index, length, and data block numbers stored in its blocks array.
8. Stop.

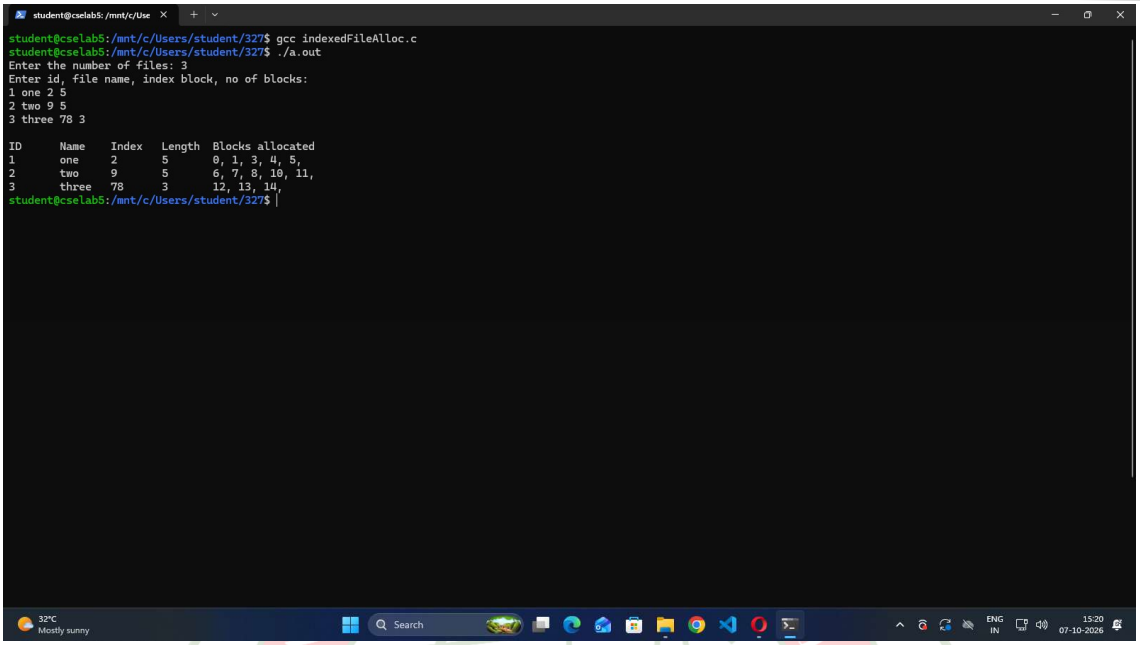
Code:

```
#include<stdio.h>
#define SIZE 100
int disk[SIZE] = {0};
struct file{
    int id;
```

```
    char name[20];
    int index;
    int length;
    int blocks[SIZE];
};
int main(){
    int n;
    printf("Enter the number of files: ");
    scanf("%d", &n);
    struct file files[n];
    printf("Enter id, file name, index block, no of blocks:\n");
    for(int i=0;i<n;i++){
        scanf("%d %19s %d
%d",&files[i].id,files[i].name,&files[i].index,&files[i].length);
        if(files[i].index < 0 || files[i].index >= SIZE || files[i].length <= 0 ||
files[i].length >= SIZE || files[i].id == 0){
            printf("Invalid file details\n");
            return 0;
        }
        if(disk[files[i].index] != 0){
            printf("Index block is not available\n");
            return 0;
        }
        disk[files[i].index] = files[i].id;
        for(int j=0;j<files[i].length;j++){
            int block = -1;
            for(int k=0;k<SIZE;k++){
                if(disk[k] == 0){
                    block = k;
                    break;
                }
            }
            if(block == -1){
                printf("Blocks are not available\n");
                return 0;
            }
            files[i].blocks[j] = block;
            disk[block] = files[i].id;
        }
    }
    printf("\nID\tName\tIndex\tLength\tBlocks allocated\n");
    for(int i=0;i<n;i++){

printf("%d\t%s\t%d\t%d\t",files[i].id,files[i].name,files[i].index,files[i].length
);
        for(int j=0;j<files[i].length;j++){
            printf("%d ",files[i].blocks[j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:



CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
స్వయం తేజస్విన్ భవ
1979

Signature of the Faculty……...